



AI GAMES

30.12.2024

Team Members

Mohamed Ibrahim

Mohamed Karim

Mohamed Mostafa

Youssef Walid

Overview

This project is a Python-based application that provides a collection of interactive games through a graphical user interface (GUI). It leverages Tkinter for GUI design, alongside algorithms for gameplay mechanics and AI-driven functionalities.

Goals

1. **Engaging User Experience**
 - Create an intuitive, interactive GUI using Tkinter.
2. **Game Implementation**
 - Develop classic games like **Towers of Hanoi**, **Connect Four**, and **Maze Solver** with efficient algorithms.
3. **AI Integration**
 - Implement AI-driven mechanics using *A Search**, **Minimax**, and heuristics for challenging gameplay.
4. **Scalability and Modularity**
 - Design a flexible codebase to easily add new games or features.
5. **Skill Development**
 - Showcase algorithmic problem-solving and AI functionalities for educational value.
6. **Future-Proofing**
 - Enable advanced features like adaptive AI, enhanced graphics, and data persistence in later iterations.

Specifications

Games Included

1. Towers of Hanoi

- **Gameplay:**
 - A classic puzzle game where the player moves disks from one rod to another while adhering to specific rules:
 1. Only one disk can be moved at a time.
 2. A larger disk cannot be placed on top of a smaller disk.
 3. All disks must be transferred to another rod in the same order.
- **Algorithm Used:**
 - *A Search Algorithm**:
 - Utilized to find the optimal solution efficiently.
 - Includes:
 - **Heuristic Function:** Estimates the cost to reach the goal based on the current state of the disks.
 - **Neighbor Function:** Generates valid moves from the current state.
- **AI Enhancements (Future):**
 - Improved heuristics for larger disk configurations.
 - Dynamic visualization of AI moves for better interactivity.

2. Connect Four

- **Gameplay:**
 - A two-player strategy game where players alternately drop colored discs into a 7x6 grid. The objective is to form a line of four discs horizontally, vertically, or diagonally before the opponent.
- **Algorithm Used:**
 - **Minimax Algorithm:**
 - Evaluates possible moves and chooses the optimal one by simulating all potential outcomes.
 - **Alpha-Beta Pruning:** Optimizes the Minimax algorithm by reducing the number of nodes evaluated in the search tree.
- **AI Enhancements (future):**
 - **Monte Carlo Tree Search (MCTS)** for smarter decision-making.
 - Adaptive AI difficulty by adjusting the search depth based on player performance.

3. Maze Solver

- **Gameplay:**
 - A single-player challenge to navigate through a maze and find the shortest path from the start to the goal.
- **Algorithm Used:**
 - *A Search Algorithm**:
 - Combines path cost and a heuristic (e.g., Manhattan distance) for efficient pathfinding.
 - **Breadth-First Search (BFS):**
 - Ensures the shortest path in unweighted mazes.

- **AI Enhancements (Future):**
 - Implement **Reinforcement Learning (RL):**
 - Train an agent using Q-learning to navigate complex mazes.
 - Visualize the exploration process for user engagement.

Key Features

1. **Interactive GUI:**
 - The application uses Tkinter to create a polished and user-friendly interface with buttons, grids, and interactive elements.
 - Dynamic transitions between menus and games.
2. **Algorithmic Efficiency:**
 - Efficient use of search algorithms for gameplay mechanics.
 - Modularized structure for reusability and scalability.
3. **Customizability:**
 - Each game can be extended with more features or difficulty levels.
 - Code structure allows for easy integration of new games.

Conclusion

This project demonstrates the integration of GUI design with algorithmic problem-solving and AI capabilities. It is a versatile foundation for further enhancements in gaming, AI training, and GUI-based applications.